Changelog — helix_proxy-0.1.0-dev-0.0.2

Revision: 1 **Last modified:** 2026-07-01T09:56:36Z **Scope:** Every change on the release surface between the previous release tag `helix_proxy-0.1.0-dev-0.0.1` (commit `3f79794`, 2026-07-01 09:24:25 +0300) and the release candidate `HEAD` (`bf38c35`, 2026-07-01 12:11:30 +0300) — 14 commits. **Authority:** Helix Constitution §11.4.151 (project-prefixed release tags), §11.4.135 (standing regression-guard suite), §11.4.44 (revision header), §11.4.65 (universal Markdown export), §11.4.6 (no-guessing).

> Read-only planning/record artefact. It documents what landed; creating/pushing the tag is a separate, operator-authorised action gated by the §11.4.40 full-suite retest (see `docs/releases/RELEASE_RUNBOOK.md`).

Release-prefix note (§11.4.151)

The release prefix resolves deterministically, resolution order:

1. `HELIX_RELEASE_PREFIX` from `.env` — **unset** (no `.env` file exists in the checkout; `.env` is gitignored per §11.4.30, its template `.env.example` carries no `HELIX_RELEASE_PREFIX`).
2. Fallback = lowercased snake_case of the project-root directory basename = **`helix_proxy`**.

Resolved prefix: `helix_proxy` → this release is tagged `helix_proxy-0.1.0-dev-0.0.2` (monotonic increment from `-0.0.1`), the SAME prefix carried by the previous tag, so `git tag -l 'helix_proxy-*'` enumerates the whole release surface. Optional hardening (runbook OB-5): declare `HELIX_RELEASE_PREFIX=helix_proxy` in a gitignored `.env` + document it in the tracked `.env.example` (§11.4.77) so the prefix is authoritative rather than dir-name-derived.

Commit inventory (previous tag → HEAD, oldest first)

#	SHA	Type	Subject
1	`9a21d94`	feat	#54 — creds-drop-ready real-VPN-egress functional-proof harness + operator runbook
2	`3308e5a`	feat	#53 — optional separate plaintext `/metrics` listener (`CONTROL_API_METRICS_ADDR`)
3	`2e834a7`	fix	BUGFIX-0013 — CONST-033 scanner false-FAIL on governance-doc export siblings (F4)
4	`f219b4d`	fix	BUGFIX-0014 — anti-bluff proxy connectivity classifier (no false-FAIL on outage, no fail-open on crashed proxy)
5	`8219669`	fix	BUGFIX-0015 — require positive flood evidence before a ddos survival PASS
6	`80bd833`	fix	BUGFIX-0016 — real benchmark regression ratchet vs a committed baseline
7	`fddd25c`	docs	letsencrypt — design + phased plan for LE HTTPS with auto-renewal + rotation
8	`9ffbc0a`	fix	BUGFIX-0017 — de-bluff comprehensive-test.sh proxy canaries (F1)
9	`39c4139`	fix	BUGFIX-0018 — close `assert_egress_ip` host-undeterminable fail-open + register standing guards (F7 + F-1)
10	`2812f48`	feat	letsencrypt — Phase-1 offline cert-analyzer + self-validation guard + Status docs (task #59)
11	`ae16f61`	docs	continuation — §12.10 resume (anti-bluff sweep complete; LE Phase 1 landed)
12	`86c1d8b`	docs	releases — release runbook + prerequisite audit for `helix_proxy-0.1.0-dev-0.0.2`
13	`f2e69e0`	feat	challenges — live proxy Challenge bank (HTTP-forward + SOCKS5 + cache)
14	`bf38c35`	feat	helixqa — proxy test bank + runner with honest §11.4.3 harness-build SKIP

---

feat

- **`9a21d94` — Real-VPN-egress functional-proof harness (#54).** A creds-drop-ready harness that, once WireGuard credentials are supplied, proves the §15 VPN-routing contract end-to-end (egress through the proxy == the expected tunnel exit AND != the host's real IP), with an operator runbook. Absent a live tunnel it SKIPs honestly (`operator_attended`, §11.4.52) — never a fabricated PASS.

- **`3308e5a` — Optional separate plaintext `/metrics` listener (#53).** New `CONTROL_API_METRICS_ADDR` config lets the control-API expose Prometheus `/metrics` on a separate plaintext listener, decoupled from the fail-closed mTLS control surface, for scrape topologies that cannot present a client cert.
- **`2812f48` — Let's Encrypt Phase-1 offline cert-analyzer (task #59).** An offline certificate analyzer (`tests/letsencrypt/cert_analyzer.sh`) that is self-validating per §11.4.107(10): it ACCEPTS every golden-GOOD certificate property and REJECTS every golden-BAD one (expired / wrong-CA / wrong-host / SAN-substring / not-due-for-renewal). Ships with a registered self-validation guard + per-feature Status docs. No network — Phase 2/3 (live issuance / auto-renewal / rotation) remain planned per the LE design.
- **`f2e69e0` — Live proxy Challenge bank (task #59).** A Challenge bank (`submodules/challenges`) exercising the real data plane: HTTP-forward proxy, SOCKS5 proxy, and cache-HIT canaries, scoring PASS only on positive captured evidence (§11.4.69).
- **`bf38c35` — HelixQA proxy test bank + runner (task #59).** A HelixQA (`submodules/helix_qa`) test bank + runner for the proxy, with an honest §11.4.3 SKIP-with-reason when the harness build is unavailable — never a PASS-by-default.

fix

All six fixes in this window are **test-suite / anti-bluff-library integrity** fixes (they make the test corpus HONEST — no product/proxy behaviour changes), each rooted per §11.4.102 and closed with §11.4.115 RED→GREEN + §1.1 paired mutation evidence (full detail in `docs/issues/fixed/BUGFIXES.md`).

- **`2e834a7` — BUGFIX-0013:** the CONST-033 no-suspend scanner false-FAILed on the `.html`/`.pdf` §11.4.65 export siblings of the governance carriers (`CONSTITUTION.md`, `AGENTS.md`, `CLAUDE.md`, `QWEN.md`, `GEMINI.md`), which legitimately quote the banned host-power literals. Made the five governance `EXCLUDE_PATHS` entries extension-agnostic prefixes; a non-governance file with a banned literal or any real script invocation still trips the scanner (gate not neutered, §11.4.120). Latent — the sibling-blindness class BUGFIX-0011 closed for the bug ledger, one layer over.
- **`f219b4d` — BUGFIX-0014:** `verify-proxy.sh` / `final-verify.sh` classified every through-proxy check as `code == expected ? PASS : FAIL`, so a momentary outage of the probed *site* false-FAILed a healthy proxy (non-deterministic, not re-runnable). Added a single client-side classifier `proxy_conn_verdict` in `tests/`

`lib/evidence.sh`: proxy-reachable → PASS; proxy-miss-but-direct-200 → FAIL (real defect, §11.4.68 not fail-open); both-fail → SKIP (external outage).

- **`8219669` — BUGFIX-0015:** `ddos_flood_suite` scored "survived the flood" PASS with no proof a flood occurred. Added a pure `flood_survival_verdict` classifier requiring positive captured flood evidence (`flood_total > 0` AND `flood_responses > 0`) before any survival PASS; zero-flood on a listening proxy → FAIL, absent proxy → honest SKIP.
- **`80bd833` — BUGFIX-0016:** the benchmark regression ratchet compared against a baseline under the gitignored `qa-results/` tree, so the comparison never persisted and every run PASSed on the absolute budget regardless of a real regression. Moved the baseline to the committed path `tests/dynamic/baselines/benchmark_p95.baseline`; seed-once + SKIP on absent, FAIL on p95 growth beyond tolerance, no auto-refresh.
- **`9ffbc0a` — BUGFIX-0017:** seven proxy canaries in `comprehensive-test.sh` still used the pre-BUGFIX-0014 blind `code != expected ? FAIL` pattern. Re-wired each through a `conn_check` wrapper onto the already-committed `proxy_conn_verdict` classifier (BUGFIX-0014), so an external outage SKIPs while a real proxy defect still FAILs. The DoH content assertion is preserved via synthetic `ANSWER`/`MISS` tokens.
- **`39c4139` — BUGFIX-0018:** `assert_egress_ip` fail-opened the hardest-to-fake VPN-routing §15 proof — when the host's real IP was undeterminable, the `egress != host` half silently collapsed, so a genuine NO-VPN (`egress == host`) case could fake-PASS. Now returns exit-2 OPERATOR-BLOCKED when `host_real` is not IP-shaped (F-1 hardened it from a two-sentinel deny-list to a positive `_evidence_ip_shaped` validator). **Also closed the F5/F6 registration gap** — see the guard audit below.

docs

- **`fddd25c` — Let's Encrypt design + phased plan** for HTTPS with auto-renewal and rotation (informs the Phase-1 analyzer that landed in `2812f48`).
- **`ae16f61` — CONTINUATION §12.10 resume** refreshed to live state: anti-bluff sweep complete, LE Phase 1 landed, Phases 2/3 in flight.
- **`86c1d8b` — Release runbook + prerequisite audit** for `helix_proxy-0.1.0-dev-0.0.2` (`docs/releases/RELEASE_RUNBOOK.md`), with the read-only GitHub/GitLab readiness audit and OPERATOR-BLOCKED items OB-1..OB-7.

test

No commit in this window carries a bare `test(` Conventional-Commits prefix; the `fix(tests): …` commits above are classified as fixes (they repair defective tests). Every fix additionally lands or wires its §11.4.135 standing guard — audited next.

---

§11.4.135 regression-guard audit

Scope: every **bugfix** on the release surface (previous tag → HEAD) — BUGFIX-0013 through BUGFIX-0018. For each: the standing guard registered in `tests/run-tests.sh` → `test_regression_guards()`, whether that guard is wired with BOTH a GREEN assertion AND a `RED_MODE=1` reproduction self-check, and any honest gap.

Evidence basis (§11.4.6 honest boundary). This audit is a documentation pass; the guards were **not re-executed here** (read-only task; the `tests/` tree is not run as part of authoring — the §11.4.40 full-suite retest is the operator-run authoritative re-run before the tag, per the runbook). The GREEN + RED status below is established from two captured sources: (a) direct read of `tests/run-tests.sh` `test_regression_guards()` (lines 473–699) confirming each guard is wired with both a GREEN invocation and a `RED_MODE=1` self-check; (b) the captured RED→GREEN + §1.1 mutation-with-md5-restore runs recorded per fix in `docs/issues/fixed/BUGFIXES.md`, including the standing-suite capture at BUGFIX-0018 time: `tests/run-tests.sh -> 59 run / 53 pass / 6 skip / 0 fail; BUGFIX-0018 GREEN+RED both PASS`.

Fix	Registered guard (`tests/regression/…`)	Wired GREEN	Wired `RED_MODE=1`	Verc
BUGFIX-0013	`no_suspend_export_sibling_test.sh` (shared; GREEN branch extended for governance-doc siblings)	yes (L572)	yes (L582)	**GUA** (sha guar
BUGFIX-0014	`proxy_conn_verdict_test.sh`	yes (L615)	yes (L625)	**GUA** (ded
BUGFIX-0015	`ddos_flood_evidence_test.sh`	yes (L635)	yes (L641)	**GUA** (ded
BUGFIX-0016	`benchmark_baseline_ratchet_test.sh`	yes (L651)	yes (L657)	**GUA** (ded
BUGFIX-0017	*(no dedicated guard)* — reuses `proxy_conn_verdict_test.sh` (BUGFIX-0014)	via 0014	via 0014	**GUA BY-I** — se
BUGFIX-0018	`assert_egress_ip_host_unknown_test.sh`	yes (L669)	yes (L675)	**GUA** (ded

Additionally, the **Let's Encrypt Phase-1** feature (`2812f48`, not a bugfix but shipped this window) registers a self-validation guard `cert_analyzer_selfvalidation_test.sh` (GREEN L687 + `RED_MODE=1` L693).

All 12 guard files referenced by `test_regression_guards()` are present on disk under `tests/regression/` (verified). Each is wired GREEN + RED (the RED self-check enforces §11.4.7 — a guard that cannot reproduce its defect is itself a finding).

Tally

- Release-window bugfixes audited: **6** (BUGFIX-0013…0018).
- With a registered standing guard (dedicated or shared), wired GREEN + RED: **5** (0013 shared, 0014, 0015, 0016, 0018).
- Guarded-by-reuse (no dedicated guard; documented; runtime proof captured): **1** (0017).
- **True unguarded GAPS: 0.**

Honest findings (§11.4.6 — not papered over)

1. **BUGFIX-0017 has no dedicated standing guard (documented reuse, not a silent gap).** Per its own ledger entry, BUGFIX-0017 only re-wires `comprehensive-test.sh`'s canaries onto the *already-guarded* classifier `proxy_conn_verdict` (covered by BUGFIX-0014's registered `proxy_conn_verdict_test.sh`, full truth-table + RED/GREEN polarity + §1.1 mutation). Its own lane-specific runtime proof is the captured conductor live smoke at `qa-results/regression/comprehensive_f1_conductor_smoke/` (CASE1 working-proxy → PASS, CASE2 simulated outage → `SKIP:network_unreachable_external`). This is a legitimate, documented reuse — the invariant IS guarded — but it is surfaced here honestly rather than counted as a dedicated guard.

2. **A real §11.4.135 registration gap existed *within* this window and was CLOSED within it.** The F5 (`ddos_flood_evidence_test.sh`, BUGFIX-0015) and F6 (`benchmark_baseline_ratchet_test.sh`, BUGFIX-0016) guards were committed to disk with their fixes but were **not wired** into `test_regression_guards()`, so for two commits (`8219669`, `80bd833`) they never ran on a build — a live gap. BUGFIX-0018 (`39c4139`) surfaced and closed it, registering F5 + F6 + F7 (each GREEN + `RED_MODE=1`). As of `HEAD` both are wired (L635/L641 and L651/L657). Net state at the release candidate: no on-disk-but-unwired guard remains among the audited set. This is recorded so the history is not smoothed over.

3. **Re-execution deferred to the release gate.** No guard was re-run in this authoring pass (see Evidence basis). The §11.4.40 full-suite retest on a clean baseline — the operator-run release gate documented in the runbook (Step 2) — is the authoritative re-run that must be GREEN before the `helix_proxy-0.1.0-dev-0.0.2` tag is created. This changelog does not assert a fresh run it did not perform.

Sources

- `docs/issues/fixed/BUGFIXES.md` (BUGFIX-0013…0018 entries — root cause, RED→GREEN, §1.1 mutation md5 restores, captured evidence paths).
- `tests/run-tests.sh` → `test_regression_guards()` (lines 473–699 — guard wiring).
- `docs/releases/RELEASE_RUNBOOK.md` (prefix resolution, tag naming, §11.4.40 gate, OPERATOR-BLOCKED items).
- `git log helix_proxy-0.1.0-dev-0.0.1..HEAD` (commit inventory).